

빠르게 만들되, 끝까지 검증하는
백엔드 개발자 류병선 입니다.

Contact

☎ 010-5564-3135 | ✉ abcfg89@naver.com



류병선

Backend Developer

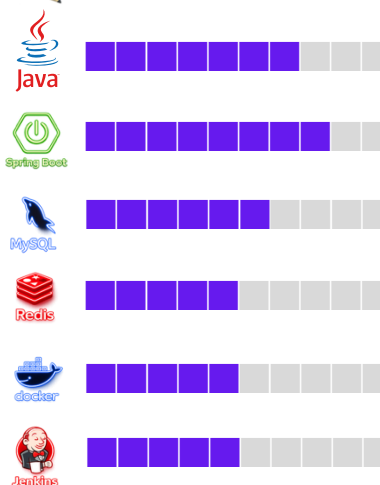
Projects

- 2025.11 ~ 2025.12 AvoFit 시기반 헬스케어 식단 서비스 | BE
- 2026.01 ~ 2026.02 CONY 기프트콘 관리/거래 플랫폼 | 거래도메인 BE
- 2026.02 ~ 2026.03 Duckchi 모임 더치페이/정산 서비스 | INFRA/인증 BE

Education

- 2019.03 ~ 2025.08 조선대학교 컴퓨터공학과 졸업 (4년제)
- 2025.07 ~ 2026.06 삼성 SSAFY 14기 Java전공 트랙

Skill



Certificate

- 2022.02 한국사능력검정시험 1급
- 2024.04 SQLD
- 2025.06 정보처리기사

Award

- 2025.12 SSAFY 14기 관통PJ 우수상 - 아보핏(AvoFit)
- 2026.03 SSAFY 14기 공통PJ 우수상 - 코니(CONY)

Projects Overview

대표프로젝트 1

Diet Coach (아보핏)

Portfolio Showcase



아보핏(AvoFit)

AI 기반 헬스케어 식단 서비스

- 한 줄 소개 AI가 사용자의 목표에 맞는 식단을 추천하는 서비스
- 담당 역할 백엔드
- 핵심 키워드 AI 추천, MyBatis, @Async, 배치 스케줄링

대표프로젝트 2



코니(CONY)

기프트콘 관리/거래 플랫폼

- 한 줄 소개 안전한 기프트콘 거래와 관리를 제공하는 플랫폼
- 담당 역할 백엔드(거래 도메인)
- 핵심 키워드 비관적 락, Redis GEO, 동시성 제어

대표프로젝트 3

덕치 (Duckchi)

모임 기반 더치페이 - 정산 서비스

자바로 로그인부터 모음 생성, 금액 등록, 정산 요청/승인, 잔액 기록까지의 연동된 서비스를 구축



덕치(Duckchi)

모임 더치페이/정산 서비스

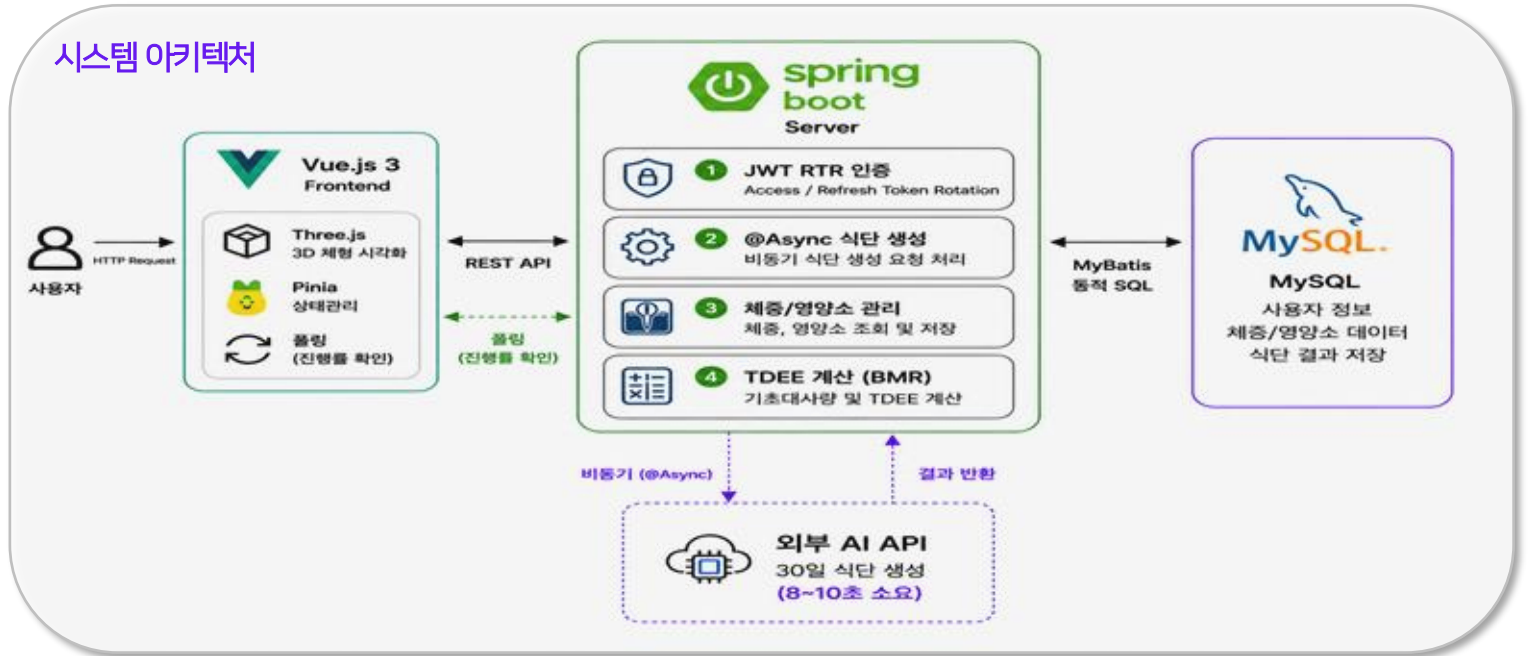
- 한 줄 소개 모임 정산과 더치페이를 간편하게 도와주는 서비스
- 담당 역할 인프라 / 인증 백엔드
- 핵심 키워드 MSA, JWT, Docker, Jenkins, Kafka

Project 1. Overview

프로젝트 기본 정보

📋 프로젝트명	: AvoFit (아보핏)
🕒 기간	: 2025.11 ~ 2025.12 (4주)
👥 팀 구성	: Full-Stack 2명
✍️ 내 역할	: 백엔드 전체 담당 인증, AI 식단 생성, 체중 관리, 비동기 처리
📚 사용기술	: Java 17, Spring Boot, MyBatis, MySQL, JWT
✨ 링크	: GitHub

시스템 아키텍처



한 줄 소개

개인 대사량(TDEE) 기반 AI 맞춤 식단을 자동 생성하고, 체형 변화를 3D로 시각화하는 헬스케어 플랫폼입니다.

이 프로젝트가 해결하는 문제

기존 식단 앱은 단순 칼로리 계산에 그쳐 개인별 대사량을 반영한 식단 추천과 동기 부여용 시각적 피드백이 부족했습니다. AvoFit은 TDEE 기반 AI 맞춤 30일 식단 자동 생성과 3D 체형 시각화로 개인화된 식단 관리 경험을 제공합니다.

개인 기여도

✅ 개인 기여도 (전체 기여도 : 50%)

AI 식단 로직 및 비동기 통신
식단 도메인 백엔드 핵심

- 외부 AI API 클라이언트 연동 및 MealPlanServiceImpl 식단 생성 비즈니스 로직 설계
- 식단 생성(8~10초 소요) 타임아웃 방지를 위한 백그라운드 비동기 통신 파이프라인 구축

장보기 연동 및 데이터 정규화
11번가 커머스 완벽 통합

- 11번가 상품 검색 API 연동 및 IngredientQueryNormalizer를 통한 식재료 검색어 정규화 처리
- ProductWeightParser를 직접 구현하여 수량/단위 텍스트를 파싱하고 실제 구매 상품 매핑 정확도 극대화

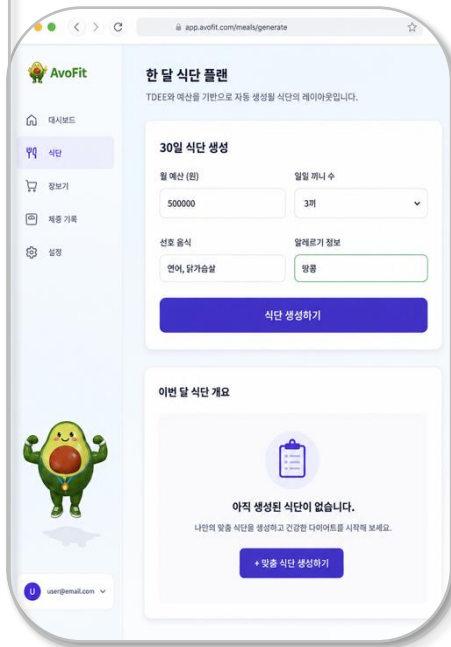
상품 스코어링 & AI 리랭킹
검색 매핑 알고리즘 고도화

- ProductScorer 설계: 예산, 용량, 단가를 복합적으로 고려한 최적 상품 자체 점수화(Scoring) 구현
- AiProductReranker 도입: 단순 키워드 검색을 넘어 컨텍스트에 맞는 후보군을 재정렬하여 장보기 리스트 품질 대폭 향상

Project 1. Core API Flow

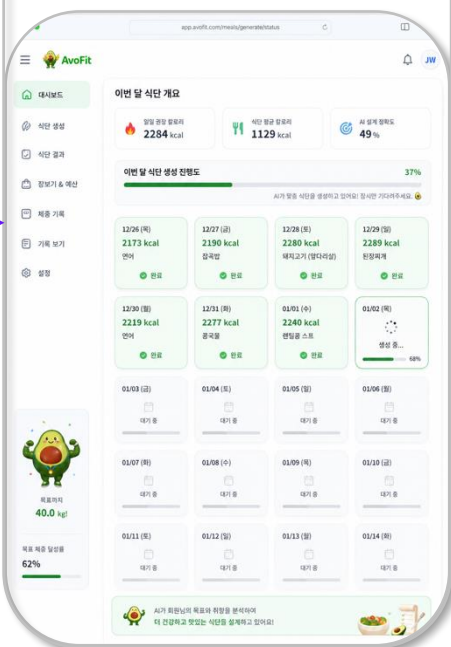
① 식단 생성 입력

예산·끼니 수·선호 음식·알레르기 정보를 입력하여 맞춤 식단 생성을 요청합니다.



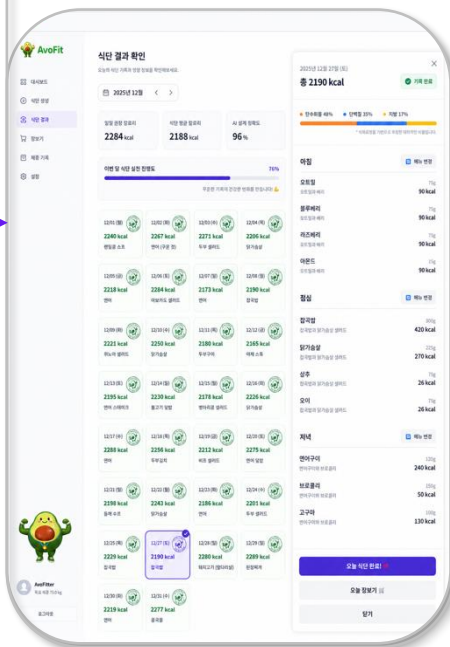
② 비동기 AI 처리

@Async 기반으로 식단 생성 작업을 비동기로 처리하고, 폴링으로 진행 상태를 확인합니다.



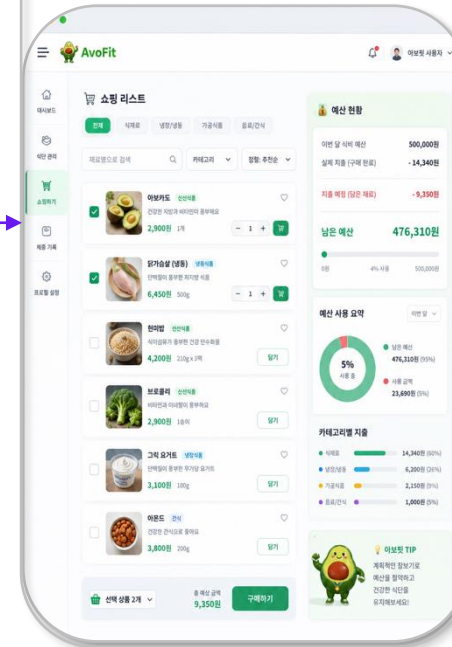
③ 식단 결과 확인

30일 식단 결과와 영양 정보를 확인합니다.



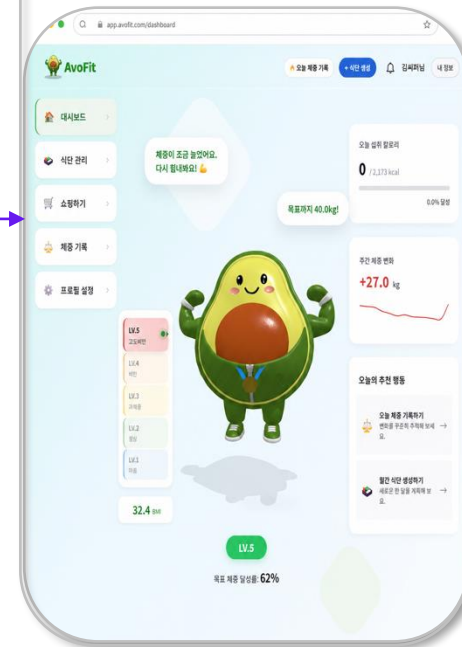
④ 장보기·11번가 연동

식단 결과를 장보기 리스트로 변환하고, 예산을 고려하여 11번가 구매로 연결합니다.

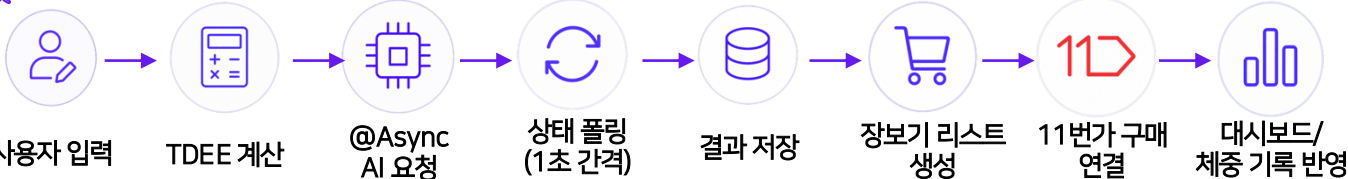


⑤ 대시보드 반영

식단/체중 기록에 반영되어 대시보드와 체중 기록 화면에서 목표 달성 현황을 시각적으로 확인합니다.



Technical Flow



Async

응답 시간 8~10초
대기 대신
즉시 요청 수락

Shopping

식단 결과를
실제 구매 행동으로
연결

Visualization

대시보드와
체중 기록으로
변화 추적

Project 1. Problem Solving 01

문제 상황

① 예산오차

AI가 30일 식단 생성 시 재료비 합계가 사용자 예산(30만원)과 크게 차이 발생
→ 예산 기반 서비스인데 예산이 안 맞음

② 상품 매핑 오류

AI가 식단 재료명("닭가슴살 100g x 30팩")을 그대로 11번가에 검색
→ 계란박스·대용량 묶음 등 엉뚱한 상품 노출

원인 분석

① AI API는 식재료 종류만 반환

시장 단가를 모르기 때문에 예산 파라미터를 전달해도 총 재료비 합계 제어 불가

② 재료명에 수량·단위·포장 정보 혼재

"닭가슴살-100g x 30팩" 그대로 검색어 사용 시 노이즈 발생
→ 원하는 단품 상품 연결 실패

해결 방법

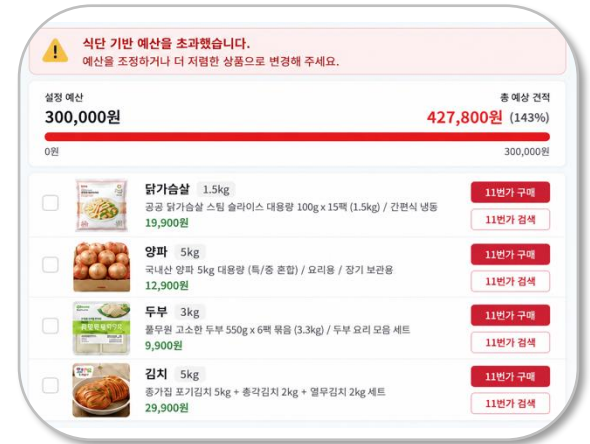
① filterItemsToBudget() — Greedy Filter

재료를 추정 단가 기준 내림차순 정렬 예산 초과 시 비싼 항목부터 순차 제거 (허용 범위 : 예산 x 1.15 이내)

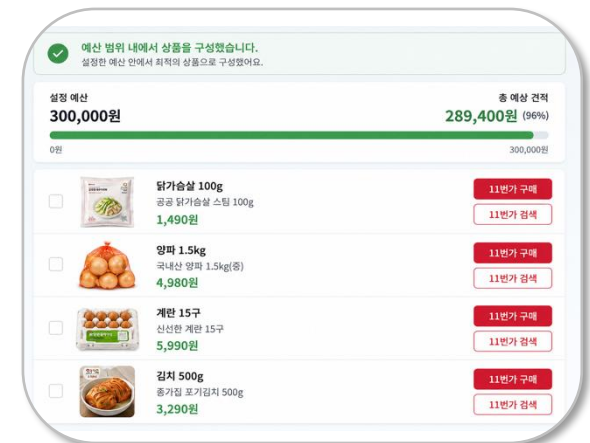
② IngredientQueryNormalizer

수량·단위·불용어 제거
"닭가슴살-100g x 30팩" → "닭가슴살"
카테고리 화이트리스트 필터링
→ 후보 TopN 스코어링
→ 3개 이상 시 AI rerank
→ 대표 상품 1개 선정

개선 전



개선 후



결과

예산 오차	:	무제한 초과	→	✅ 예산 ±15% 이내로 제어
상품 정확도	:	엉뚱한 상품 노출	→	✅ 키워드 정규화로 매핑 정확도 개선
API 안정성	:	실패 시 MOCK Fallback으로 서비스 중단 없이 기본 응답 제공	→	✅ 안정적인 API 운영 & 사용자 경험 보장

Project 1. Problem Solving 02

문제 상황

외부 AI API가 식단 생성에 8~10초 소요

→ 동기 처리 시 브라우저 타임아웃(30초) 임박

→ 사용자 응답 없음으로 인식
서비스 이탈 발생

원인 분석

식단 생성 요청이 동기적으로 처리되어

AI API 응답이 올 때까지

스레드 블로킹 클라이언트는 응답 대기 중

아무것도 할 수 없는 상태

해결 방법

@Async + CompletableFuture
비동기 전환

- ① 요청 즉시 200 응답 반환
- ② 백그라운드에서 AI API 처리
- ③ 프론트 폴링으로 완료 확인
(1초 간격 GET 요청)

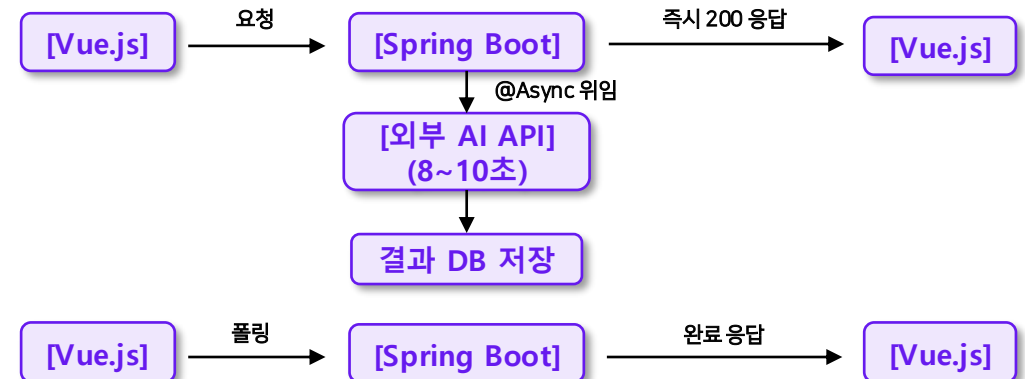
결과

응답 시간 : 8~10초 → **즉시(3초 이하)**  80% 개선

타임아웃 에러율 : 발생 → **비동기전환으로 해결**  구조적 해소

사용자 경험 : 대기 화면 → **진행률 표시로 자연스러운 UX**

처리 흐름



Project 1. Technical Decisions & Retrospective

? 기술 선택 이유



MyBatis (JPA 대신)

- 식단·영양소를 기간·조건별로 집계하는 복잡한 쿼리가 핵심
 - JPA는 동적 조건 쿼리 제어 한계 + N+1 이슈 우려
- 동적 SQL + resultMap으로
실사용 흐름에 맞게 직접 설계



JWT RTR (일반 JWT 대신)

- Refresh Token은 수명이 길어 탈취 시 장기간 악용 가능
- 사용 즉시 기존 토큰 폐기,
재사용 시 즉각 무효화
- 보안 취약점을 구조적으로 차단



@Async (WebFlux 대신)

- Spring MVC 기반 구조에서 WebFlux 전환 시 전체 리팩토링 비용 + 러닝커브 과다
- 기존 구조 유지하며
비동기 처리만 분리



프로젝트 성과



기술적 성과

- @Async 도입 → 응답시간 80% 개선, 타임아웃 이슈 구조적 해소
- 예산 필터링 도입 → 장보기 비용 ±15% 이내 제어



구현 범위

- 인증·식단·장보기·체중·대시보드 백엔드 전체 단독 설계 (BE 1인, 4주)
- 11번가 실제 커머스 API 연동 완료 (REAL/MOCK Fallback으로 API 장애 시 기본 응답 제공)



협업

- GitFlow + PR 단위 코드 리뷰로 FE-BE 인터페이스 충돌 없이 완료



회고



아쉬운 점

- 상품 파싱 실패 케이스 일부 잔존 (수량·단위 혼재 패턴 완전 미해결)
- 단위 테스트 미작성 → 리팩토링 시 회귀 검증 불가



배운 점

외부 API를 붙이는 것보다
실패·타임아웃·사용자 대기 상태까지
고려한 흐름 설계가 더 중요하다



개선 방향

- AI rerank 후보 품질 강화 (하드필터·스코어링 먼저 개선)
- 파싱 정규화 화이트리스트 확장
- Redis 캐싱으로 11번가 API 호출 최소화

Project 2. Overview

프로젝트 기본 정보

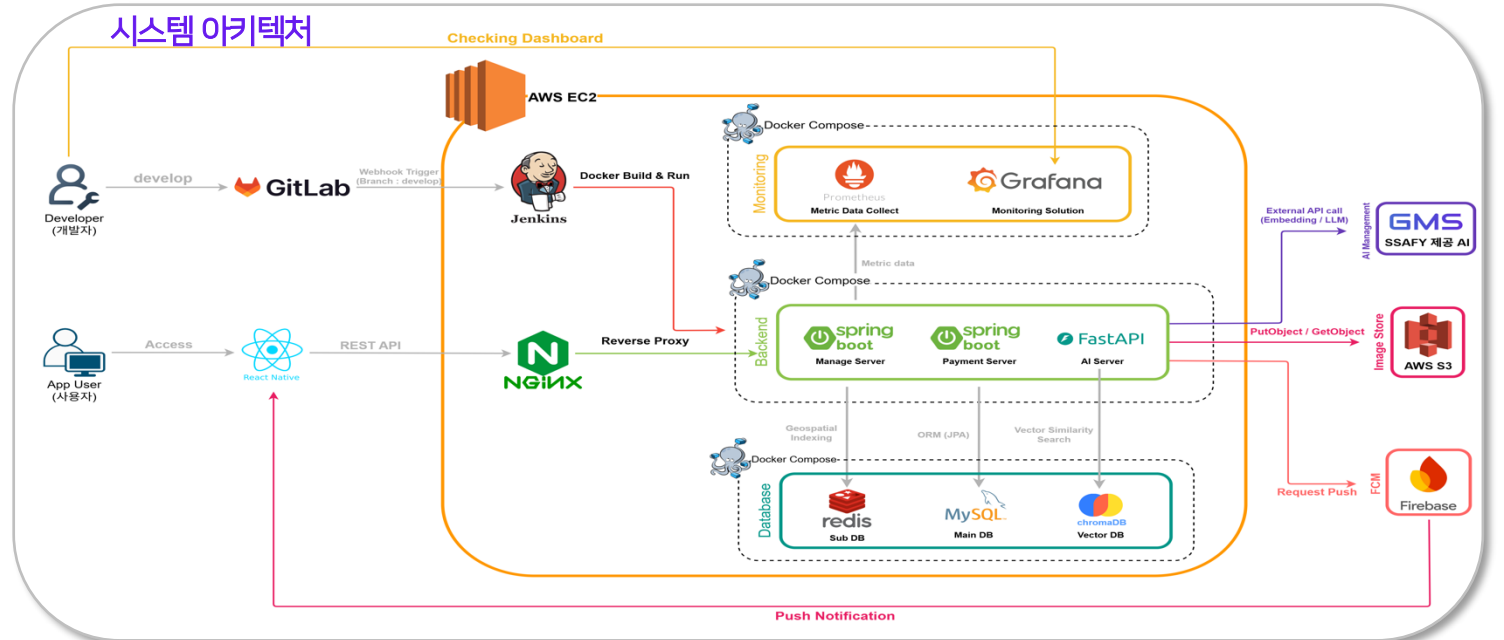
📅 프로젝트명	: CONY (코니)
🕒 기간	: 2026.01 ~ 2026.02 (6주)
👥 팀 구성	: FE 2명 / BE 2명 / AI 1명 / INFRA 1명
🔧 내 역할	: 백엔드 (거래 도메인 전담) 구매·판매·포인트 차감·위치 검색·배치 자동화
📚 사용기술	: Java 17, Spring Boot, JPA, Redis, MySQL
🌐 링크	: GitHub

한 줄 소개

AI OCR 자동 등록부터 위치 기반 알림, 안전한 에스프로 거래까지 기프트콘 자산을 스마트하게 관리하는 통합 플랫폼입니다.

이 프로젝트가 해결하는 문제

기프트콘은 유효기간, 사용 가능 매장, 보유 여부를 사용자가 직접 관리해야 해 미사용·기간 만료로 인한 손실이 쉽게 발생합니다.
CONY는 AI OCR 자동 등록, 위치 기반 사용 알림, 유효기간 임박 자동 판매 기능을 통해 기프트콘을 단순 보관이 아닌 사용·거래·공유까지 이어지는 자산 관리 흐름으로 개선했습니다.



개인 기여도

✅ 개인 기여도 (전체 기여도 : 33%)

거래 시스템 (구매·판매·비관적 락)

BE 기여도 중 핵심

구매 3단 검증 (상태·본인·포인트),
PESSIMISTIC_WRITE 락으로 동시성 제어,
소유권 이전까지 하나의 트랜잭션으로 처리

결제 연동 (카카오페이)

결제 로직 단독 구현

카카오페이 API 클라이언트 단독 구현,
결제 흐름 내 JWT 인증 통합,
리다이렉트 및 결제 승인 처리 전체 구현

배치 자동화 & 위치 검색

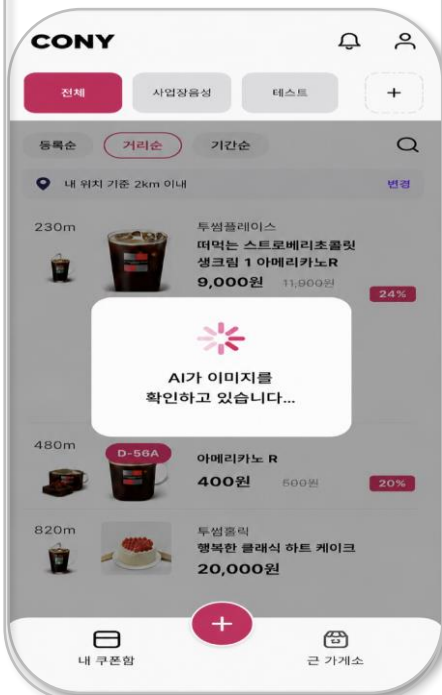
해당 기능 100%

(Spring Scheduler)
매일 09:00 임박 상품 자동 판매 등록,
(Redis GEO)
GEORADIUS 반경 검색 및 거리순 정렬 구현

Project 2. Core API Flow

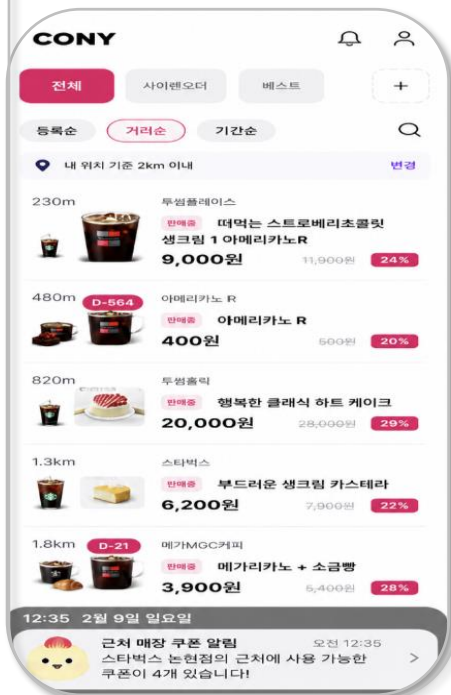
① AI 자동 등록 (OCR)

이미지 업로드 시 AI 서버가 브랜드, 상품명, 유효기간, 바코드를 자동 추출하여 기프티콘을 등록합니다.



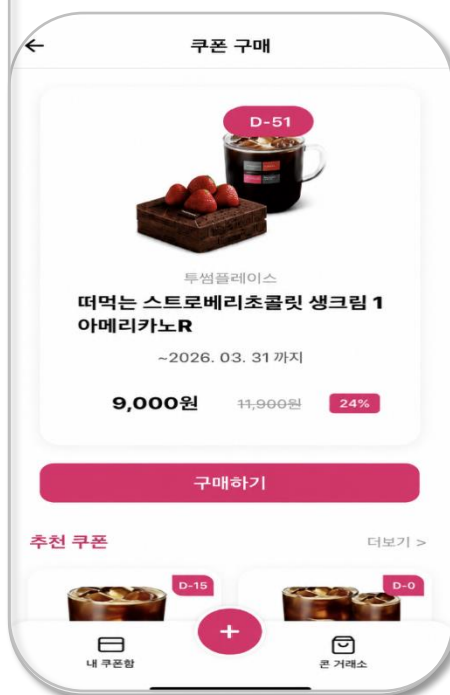
② 위치 기반 판매글 검색

사용자 주변 매장을 Redis GEO를 활용해 반경 내 거리순으로 빠르게 탐색하고 추천합니다.



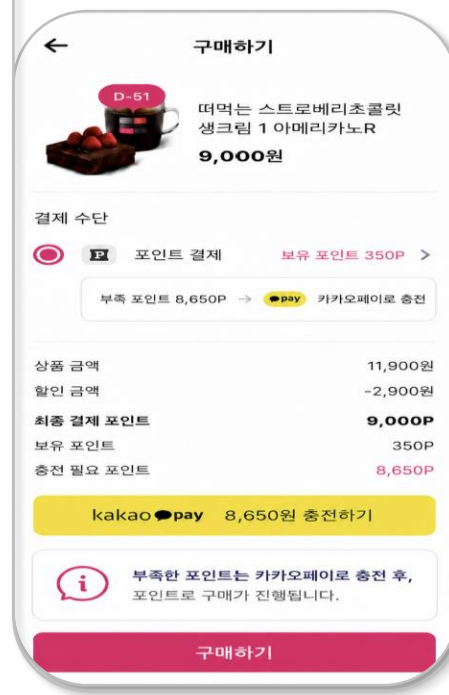
③ 안전한 에스스로 거래

동시 구매 방지를 위해 DB 레벨의 비관적 락(Pessimistic Lock)을 적용하였습니다.



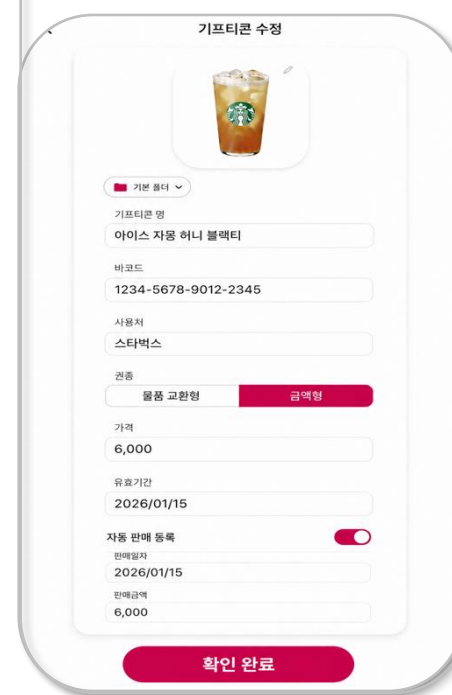
④ 결제 및 소유권 이전

카카오페이 및 포인트 결제 후, 판매 상태 변경 및 소유권을 안전하게 이전합니다.



⑤ 배치 자동화 (Scheduler)

매일 09:00, 유효기간 임박 기프티콘을 시스템이 자동으로 최적이 판매글을 등록합니다.



Technical Flow



⌚ 동시성 제어

비관적 락 적용
동시 구매 요청 시
중복 결제 원천 차단

⚡ 검색 최적화

Redis GEO 활용
MySQL 대비 위치/거리 연산 속도 대폭개선

🤖 운영 자동화

사용자 설정 기반
Spring Scheduler로
만료 임박 상품
자동 판매 등록 및 관리

Project 2. Problem Solving 01

문제 상황

사용자 위치 기반 "가까운 매장순"
판매글 정렬 시
MySQL에서 Haversine 공식으로
모든 매장과 거리 계산
→ 매장 수 증가 시 $O(N)$ 풀스캔 + 수학 연산
→ 응답 속도 저하 (매장이 많아질수록 느려짐)

원인 분석

MySQL의 한계

- ① 매 요청마다 전체 매장에 대해 Haversine 삼각함수 연산 수행 (sin, cos, acos)
 - ② 거리 계산 결과 정렬 시 인덱스 활용 불가
 - ③ 브랜드별 최소 거리 계산에 GROUP BY + 서브쿼리 필요
- 위치 검색 요청이 많아질수록 DB 부하 급증

해결 방법

Redis GEO로 위치 데이터를
인메모리 캐시 계층에 분리

- ① GEOSEARCH로 반경 5km 내 매장만 조회
→ 원거리 매장 연산 자체를 제거
- ② Pipeline으로 브랜드명 일괄 조회
→ N번 왕복 대신 1회 요청으로 처리
- ③ 브랜드별 최소 거리 맵 생성 후 메모리 정렬
→ DB GROUP BY 없이 경량 처리

결과

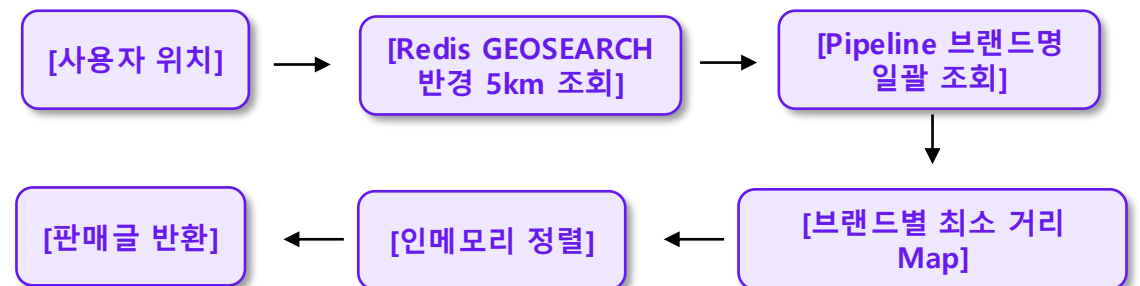
Before (MySQL)

After (Redis GEO)

거리 계산	:	매 요청마다 전체 매장 풀스캔	✅ 반경 내 매장만 조회
정렬 방식	:	DB ORDER BY (인덱스 불가)	✅ 인메모리 정렬
네트워크	:	매장 N개 × N번 왕복	✅ Pipeline 1회
확장성	:	매장 증가 시 성능 저하	✅ 캐시 계층 분리로 DB 부하 차단

✅ 위치 기반 검색 성능 개선 및 DB 부하 감소

처리 흐름



Project 2. Technical Decisions & Retrospective

기술 선택 이유



JPA 비관적 락 (PESSIMISTIC_WRITE)

- 금융 거래 특성상 충돌 시 재시도보다 확실한 순차 처리가 중요
- "단 한 건의 중복 결제도 없어야 한다"는 요구사항에 맞춰 DB 레벨의 강력한 동시성 제어 선택

→ 중복 결제를 원천 차단하는 안정적 거래 처리



Redis GEO & Pipeline

- MySQL의 Haversine 폴스캔 연산 한계를 극복
- 인메모리 기반 반경 검색과 파이프라인으로 네트워크 왕복을 최소화 하여 검색 속도 극대화

→ 위치 검색 성능과 확장성 동시 확보



Spring Scheduler

- 예약 판매 등록, 만료 상품 삭제 등 정기적인 배치 작업을 별도 프레임워크 없이 경량으로 자동화

→ 운영 자동화로 관리 비용 절감



프로젝트 성과



기술적 성과

- **결제 트랜잭션 무결성 확보** → 비관적 락 적용으로 트랜잭션 경합 시 발생할 수 있는 중복 결제 원천 차단
- **조회 성능 및 운영 효율화** → Redis 연동으로 거리 계산 병목을 완화하고, 만료 상품 관리 프로세스 자동화



구현 범위

- 카카오페이 결제, 위치 검색, 배치 자동화 등 거래 도메인 핵심 로직 단독 설계



협업

- Jira / GitLab을 활용한 체계적인 이슈 트래킹
- PR 단위 코드 리뷰를 통해 팀원 간 로직 충돌 및 FE-BE 인터페이스 문제 사전 방지



회고



아쉬운 점

- 락 대기가 길어질 경우 대규모 트래픽에서 처리량(TPS) 하락 우려
- JMeter 등을 활용한 동시성 및 부하에 대한 정량적 수치 검증 부재



배운 점

결제 도메인에서는 **빠른 응답** 보다 **정확하고 안전한 데이터 처리**가 최우선 설계 원칙임을 체감



개선 방향

- DB 부하 완화를 위한 Redis 분산 락(Redisson) 전환 검토
- k6/JMeter 기반 부하 테스트 시나리오 구축 및 병목 구간 TPS 측정

Project 3. Overview

프로젝트 기본 정보

 프로젝트명	:	Duckchi (덕치)
 기간	:	2026.02 ~ 2026.03 (5주)
 팀 구성	:	FE 1명 / FullStack 4명 / INFRA 1명
 내 역할	:	INFRA / 인증 백엔드 전담 (MSA 구조 설계 · Gateway JWT 검증 · 카카오 로그인 · 배포/모니터링)
 사용기술	:	Java 17, Spring Boot, Spring Cloud Gateway Docker Compose, Jenkins, Nginx, Prometheus
 링크	:	GitHub

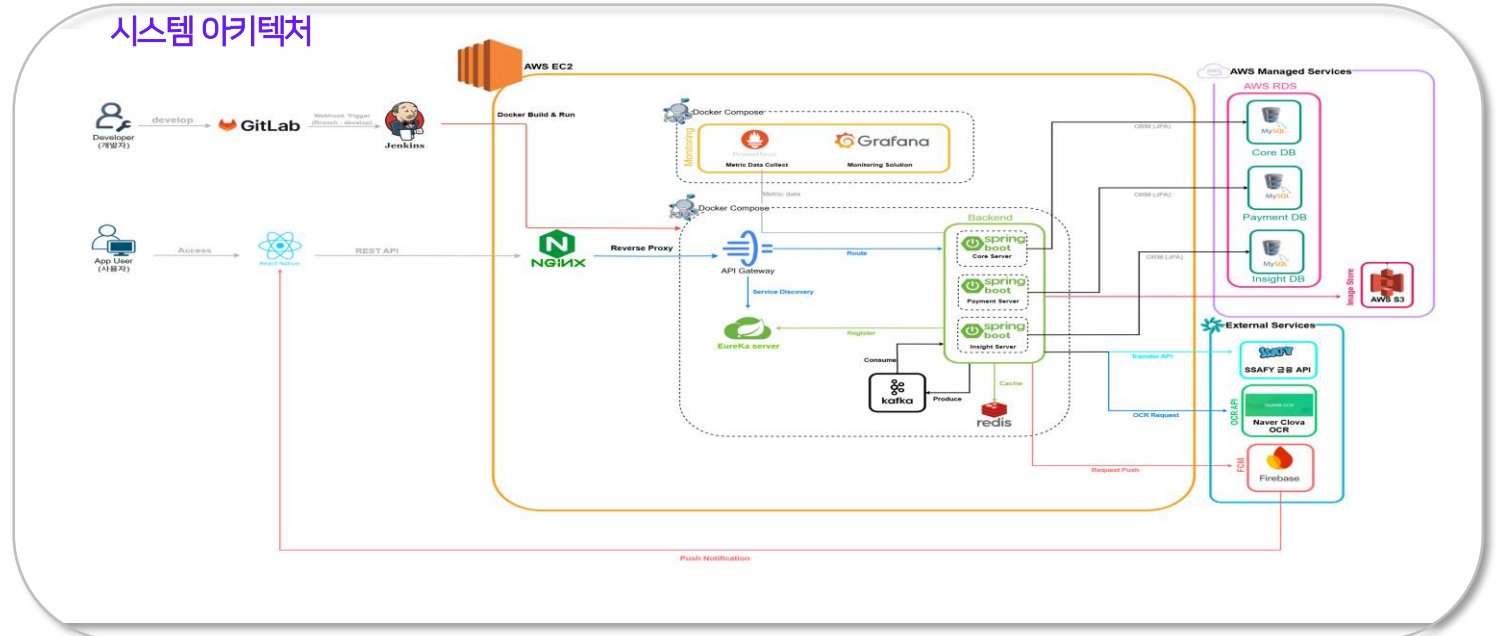
한 줄 소개

모임방 생성부터 결제 내역 등록, 자동 분할 정산, 개인별 소비 리포트까지 복잡한 정산 과정을 한 흐름으로 해결하는 모바일 더치페이 서비스입니다.

이 프로젝트가 해결하는 문제

기존의 카카오톡 개별 송금이나 엑셀 기록 방식은 누가 얼마를 냈는지 추적이 어렵고, 정산 누락이 빈번하게 발생합니다. 덕치(Duckchi)는 모임방 내역을 통합 관리하여 복잡한 분할 계산과 송금 누락 문제를 해결하고, 개인별 소비 리포트까지 제공하여 정산의 처음부터 끝까지 한 번에 이어지는 매끄러운 경험으로 개선했습니다.

시스템 아키텍처



개인 기여도

개인 기여도 (전체 기여도 : 40%)

MSA 인프라 구축 (Docker/Jenkins)

인프라 전담 (해당 부분 100%)

- Docker Compose로 5개 마이크로서비스 및 Redis, Kafka 배포 환경 일괄 구성
- Jenkins 파이프라인(develop/release 분리) 구축으로 CI/CD 배포 자동화 달성

API Gateway 인증 및 보안

인증 로직 단독 구현

- 카카오 OAuth JWT(Access/Refresh) 흐름을 Spring Cloud Gateway에 통합하여 중앙 제어
- Nginx 리버스 프록시 및 HTTPS (TLS) 운영 적용

실시간 모니터링 환경

운영 안정성 확보

- Prometheus와 Grafana를 연동한 모니터링 대시보드 구축
- 배포 후 각 서비스의 CPU, Memory, HTTP 트래픽 상태 실시간 시각화

Project 3. Core API Flow

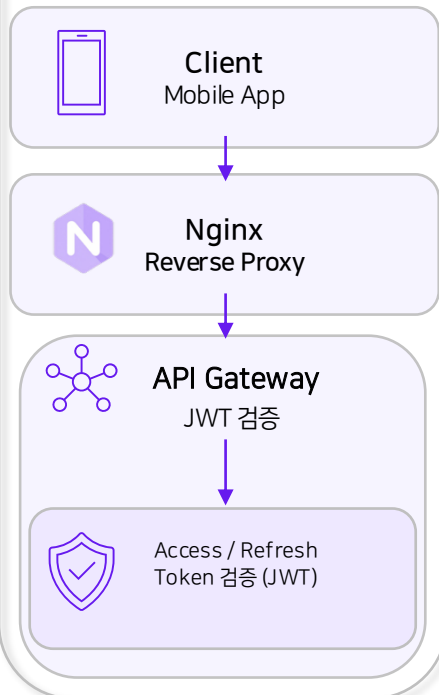
① 카카오 로그인

카카오 OAuth 2.0 연동 후 JWT 기반으로 사용자 인증을 시작합니다.



② Gateway 인증 검증

요청 진입 시 Access Token과 Refresh Token을 중앙에서 검증하고 라우팅합니다.



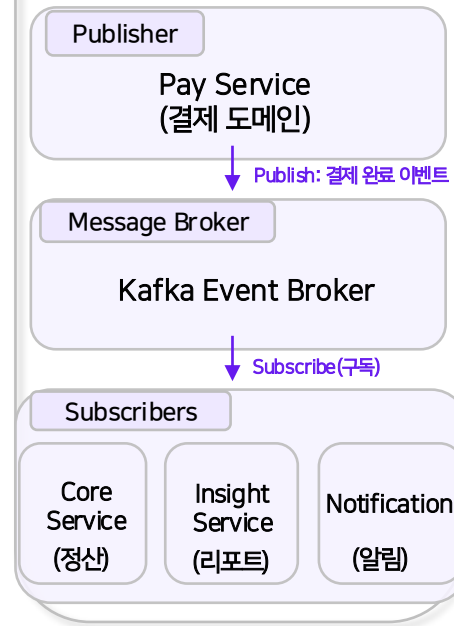
③ MSA 컨테이너 배포

Core, Pay 등 분리된 5개의 마이크로서비스를 Docker Compose로 묶어 일관된 환경으로 배포 및 운영합니다.



④ 비동기 이벤트 통신

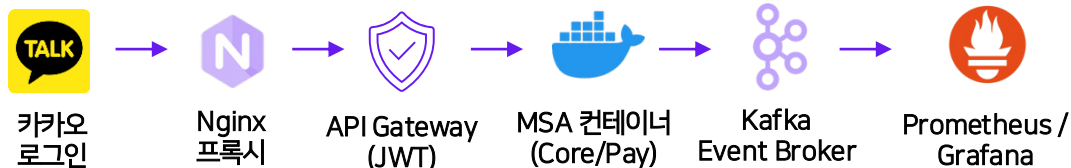
결제 및 리포트 서비스 간의 비동기 통신이 가능하도록 Kafka 메시지 브로커 인프라를 구축 및 연동하여, MSA 시스템의 강결합을 해소하고 장애 전파를 방지했습니다



⑤ 실시간 시스템 모니터링

Spring Boot Actuator와 Prometheus를 연동하여 트래픽 및 서버 리소스 상태를 실시간으로 시각화합니다.

Technical Flow



통합 인증망 구축

Gateway + Redis
JWT 중앙 검증으로 각 서비스의 인증 중복 구현을 줄임

CI/CD 배포 자동화

Docker Compose + Jenkins 5개 MSA 서비스의 빌드부터 배포까지 파이프라인 자동화

비동기·운영 모니터링

Kafka + Prometheus + Grafana 결제/리포트 이벤트 비동기 분리 및 실시간 리소스 모니터링 확립

Project 3. Problem Solving 01

문제 상황

- 1 운영 환경(EC2)에서만 API 장애 발생**
로컬에선 정상 동작하던 '모임방 생성(POST)' API가 배포 후 지속 실패 (405 Method Not Allowed)
- 2 HTTP 메서드 및 Body 소실 현상**
서버 로그 확인 결과, 클라이언트의 POST 요청이 서버 도달 직전 GET으로 강제 변환되고 Payload(Body 데이터)가 전부 날아감

원인 분석

- 1 Nginx 301 상태 코드의 스펙 한계**
Nginx에서 보안(HTTPS) 강제 전환을 위해 301 Moved Permanently 리다이렉트 사용 중
- 2 프록시 계층 규약 충돌**
일부 클라이언트 환경에서 301 응답을 받고 재요청 시, 기존 POST 메서드를 GET으로 변환하고 Body를 버리는 HTTP 스펙 상의 종속성 문제 확인

해결 방법

- 1 308 Permanent Redirect로 전면 교체**
→ Nginx nginx.conf 설정 파일 수정
→ return 308 https://\$host\$request_uri; 적용
- 2 최신 HTTP 스펙 규약 활용**
308 코드를 통해 리다이렉트 발생 시 기존 HTTP 메서드(POST)와 Body 데이터를 원본 그대로 유지하도록 네트워크 프록시 흐름 강제

결과

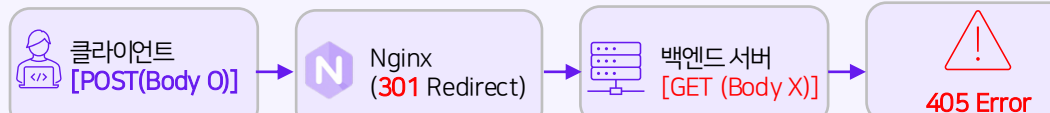
Before (301 Redirect)

After (308 Redirect)

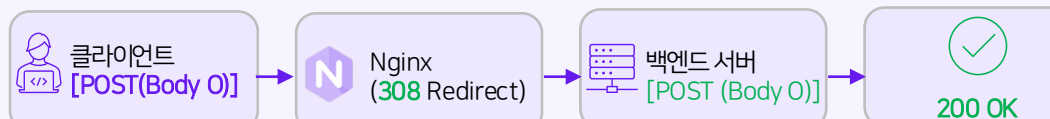
HTTP 메서드	:	POST → GET으로 변환됨	✓ POST 메서드 유지
Body 데이터	:	페이로드 소실	✓ Body 데이터 정상 전달
API 안정성	:	배포 후 통신 장애 발생	✓ 네트워크 프록시 환경 정상 동작 보장

처리 흐름

✗ 개선 전 (301)



✓ 개선 후 (308)



Project 3. Technical Decisions & Retrospective

기술 선택 이유



Docker Compose & Jenkins (수동 배포 대신)

- MSA의 5개 서비스 분리로 배포 복잡도 극대화

→ CI/CD 파이프라인 자동화로
개발/운영 환경을 분리하고
배포 과정을 안정화



Spring Cloud Gateway (각 서비스 개별 인증 대신)

- 마이크로서비스마다 인증 로직이
중복되는 문제 방지

→ JWT 전역 필터 적용으로
통합 인증망 및 단일 진입점
(Single Point) 라우팅 확보



Kafka Event Broker (API 동기 호출 대신)

- 결제와 리포트 도메인이 동기
통신 시 장애 연쇄 다운 우려

→ 비동기 메시지 큐 분리를 통해
서비스 강결합을 끊고 시스템
안정성 향상



프로젝트 성과



기술적 성과

- Jenkins 도입 → 수동 배포 과정의 휴먼 에러를 방지하고
배포 시간 단축
- Prometheus/Grafana → 실시간 리소스 및
API 트래픽 모니터링 체계 확립



구현 범위

- 6인 팀 중 인프라 전체 및 Gateway 통합 인증망 단독 전담



운영 이슈 해결

- Nginx 308 라우팅 오류, 외부 금융 API 시간대(UTC/KST) 불일치 등
운영 환경 장애 신속 디버깅 및 해결



회고



아쉬운 점

- 단일 EC2 환경 운영으로 인한 트래픽 부하 분산 한계 (Scale-out 제약)
- 서킷 브레이커(Resilience4j) 미적용으로 서비스 지연 대비 로직 미흡



배운 점

"로컬에선 되는데 운영에선 안 된다"는 이슈의 원인은 백엔드 코드보다 프록시,
네트워크 흐름(Nginx)에 있음을 깨닫고 패킷 흐름 추적 및 인프라 디버깅
역량을 길렀습니다.



개선 방향

- Kubernetes(EKS) 전환으로 오토스케일링 및 셀프 힐링(Self-Healing) 적용 검토
- Blue/Green 기반 무중단 배포 파이프라인 고도화

Why me?

1 "왜?"를 먼저 묻는 습관

기술을 선택할 때
"왜 이것이어야 하는지, 대안은 없는지"를
항상 먼저 비교하고 결정합니다.



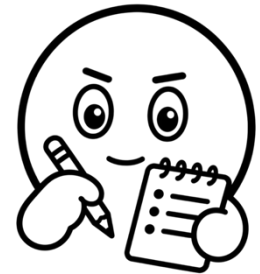
2 모르는 것을 모른다고 말할 수 있습니다.

아는 척하며 넘기지 않고,
빠르게 공유하고 함께 해결하는 것이
팀에 가장 도움이 된다고 생각합니다.



3 부족한 점을 기록하고 개선 방향을 찾습니다.

프로젝트가 끝나면 잘한 점뿐 아니라
못한 점과 다음에 어떻게 개선할지까지
반드시 함께 정리합니다.



"기술을 **선택한 이유**를 설명할 수 있고, 모르는 것을 **숨기지 않으며**, 부족한 점에서 **다음 성장 방향**을 스스로 찾는 신입 개발자입니다."

Plan

↗ 성장 방향

- 1 AI를 똑똑하게 활용하되, 맹신하지 않습니다.**
생성형 AI로 개발 생산성을 높이되,
코드의 동작 원리와 인프라 흐름은 끝까지 끈질기게 검증하겠습니다.
- 2 AI가 찾아주지 못하는 에러 앞에서도 포기하지 않는
집요함을 보여드리겠습니다.**
AI가 답하지 못하는 장애도, 직접 로그를 파헤쳐 기어코 근본 원인을
찾아내는 끈기를 증명하겠습니다.
- 3 스펀지처럼 흡수하는 '신입의 무기'를 적극 활용하겠습니다.**
빠르게 변하는 AI 트렌드와 회사의 비즈니스 도메인을 가장 먼저 배우고 적용하
여, 팀의 개발 흐름에 가장 빠르게 녹아드는 팀원이 되겠습니다.



Contact / Link



Email

abcfg89@naver.com



Phone

010-5564-3135



GitHub

github.com/ryubyeongsun



Portfolio

bs_00.github.io

현상에 타협하지 않고,
장애의 근본 원인을 끝까지 파헤치는
백엔드 엔지니어 류병선입니다.

- 감사합니다 -

Contact

☎ 010-5564-3135 ✉ abcfg89@naver.com